

Activity 4 - Vibe-code a command parser

AI Vibe Coding with Rust / TGS-2023039924 / version 3.0 / K2/A2

Specification

`parse_command(&[&str]) -> Result<Command, String>`. Accepts exactly: `list` | `add <sku> <qty>` | `remove <sku> <qty>`. `qty` must parse as `u32` and be at least 1. Anything else returns `Err` with a usage or reason message.

Plan before code

Act as a senior Rust reviewer. Specification: `parse_command(&[&str]) -> Result<Command, String>`. Accepts exactly: `list` | `add <sku> <qty>` | `remove <sku> <qty>`. `qty` must parse as `u32` and be at least 1. Anything else returns `Err` with a usage or reason message. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Vibe-code a command parser" in `starter/src` so that the supplied tests pass. Generate the parser from the command grammar only; reject any suggestion that adds crates such as `clap`. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: `<paste the first error or failing test>`. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no `unsafe`; no `unwrap()` on input; no new crates; every boundary tested; cargo test, `clippy -D warnings` and `fmt --check` pass; you can explain every line.